

FSD - Lecture 2 Notes

DOUBT LINK :

<https://docs.google.com/forms/d/e/1FAIpQLSdQWT2nyvN8mKoXtq7CgQ4VOd7ndh8EACmBdcSvUxkbsJNNVw/viewform?usp=publish-editor>

JavaScript Introduction

JavaScript (JS) is a **high-level, interpreted, or just-in-time compiled** programming language.

- **Single-threaded:** It can only do one thing at a time.
- **Non-blocking:** It uses an "Event Loop" to handle tasks like API calls without freezing the UI.
- **Evolution:** Originally created only for browsers (Frontend), but thanks to **Node.js**, it can now run on servers (Backend).

The V8 Engine

V8 is Google's open-source high-performance JavaScript and WebAssembly engine, written in C++. It is used in **Chrome** and **Node.js**.

- **The Mission:** It translates JS code directly into **Machine Code** (binary) that the computer's processor can understand.
- **Just-In-Time (JIT) Compilation:** Unlike older languages that compile all at once or interpret line-by-line, V8 compiles code *during* execution to optimize performance.
- **Key Components:**
 - * **Ignition (Interpreter):** Generates bytecode.
 - **Turbofan (Optimizing Compiler):** Turns bytecode into highly optimized machine code based on data patterns.

The core concepts to learn for MERN

1. *Variables*
2. *Functions*
3. *Events*
4. *The Document object model (DOM)*

..

How it Works

Stage	Process	Output
Parsing	Breaking code into tokens and structure	Abstract Syntax Tree (AST)
Interpretation	Turning AST into quick-to-start code	Bytecode
JIT Compilation	Optimizing "Hot" code while running	Optimized Machine Code
Execution	The CPU runs the code	Final Result / UI Update

Here is the explanation using a single code snippet and a one-line summary for each stage of the **JIT** process.

```
function add(a, b) {  
  return a + b;  
}
```

```
for (let i = 0; i < 1000; i++) {  
  add(5, 10); // Running the same task repeatedly  
}
```

The JIT Workflow in Single Lines

- **Parsing:** The engine converts your code into an **AST** (a logical map) to understand your intent.
- **Interpreting (Ignition):** The engine quickly turns the AST into **Bytecode** to start running the function immediately.
- **Profiling:** A "watcher" notices the `add` function is being called 1000 times with **Numbers** and marks it as "Hot."
- **Compiling:** The JIT compiler turns that hot bytecode into **Optimized Machine Code** (\$0101\$) to run at maximum hardware speed.
- **Bailing Out:** If you suddenly call `add("Hi", "Bye")`, the engine discards the "Number-only" optimization and goes back to the slow interpreter to stay safe.

Where JS Output is Shown

Depending on the environment, you see JS output in different places:

- **Browser Console:** Used for debugging. Accessed via `F12` or `Inspect > Console`.
- **The DOM:** Outputting data directly onto the webpage (e.g., changing text in a `<div>`).
- **Alert Boxes:** Using `alert()`, though rarely used in modern apps.
- **Terminal/Command Prompt:** When using **Node.js**, output is shown in your system's terminal.

DOM Structure

DOM (Document Object Model) is a programming interface for web documents. It represents the page as a **Tree of Objects**.

- **The Root:** The `window` object is the global parent.
- **The Document:** `window.document` is the gateway to the HTML content.
- **Nodes:** Every HTML tag is an **Element Node**, every piece of text is a **Text Node**, and every comment is a **Comment Node**.

..

DOM Manipulation in Console

You can experiment with JS directly in the browser console to change a website in real-time.

A. Selecting Elements

// By ID

```
let title = document.getElementById('header');
```

// By Class (Returns a list)

```
let buttons = document.getElementsByClassName('btn');
```

// Modern way (The best way)

```
let nav = document.querySelector('.navbar');
```

```
let allItems = document.querySelectorAll('li');
```

B. Changing Content & Style

// Changing Text

```
title.innerText = "Welcome to My Site";
```

// Changing HTML

```
document.body.innerHTML = "<h1>Hacked!</h1>";
```

// Changing Style

```
title.style.color = "blue";
```

```
title.style.backgroundColor = "yellow";
```

C. Adding/Removing Elements

// Creating an element

```
let newDiv = document.createElement('div');
```

```
newDiv.innerText = "I am new here";
```

// Adding to the page

```
document.body.appendChild(newDiv);
```

// Removing

```
newDiv.remove();
```